硬件接线与上电前检查清单

> **这份清单的目的：避免烧模块、烧 FPGA，以及避免在"数据全是错的"上耗掉一周。**

---

0. 一句话结论（★ 已按原理图逐针量取定案）

> **AD9238 载板插在开发板的 `J6` 扩展口上**（Header_2*26，Bank 16）。 **输出侧（ABZ / UVW / SPI / 中断）走 `J5` 扩展口**（Header_2*26，Bank 15）。 引脚表已经写进 `constr/enc_top.xdc`，并可由**两个**脚本复现校核（都 exit 0）： `python tools/pinout_verify.py -> RESULT: PASS （50/50 端口体检，A–G 七项） python tools/pinplan_check.py -> RESULT: PASS （论证：为什么不能照抄厂商表）` **上电前必须把跳线 `J10` (VCCIO_B16) 与 `J9` (VCCIO_B15) 都拨到 3.3 V。**

0.1 引脚方案是怎么定下来的（可复现的三方交叉验证）

不再靠"看图找针号"，而是把多份权威文件机器对齐：

输入	文件	提供什么
① 载板映射表	`AD9238_carrier_pin_map.xls` 工作表 `XC7A35A100V2.0`	厂商给出的同一载板在 **J6 / J7 / J11 三个插位**上的**三套互斥** FPGA 引脚映射（28 个信号）
② 开发板占用表	`DLC35_io_occupancy.xlsx`	本板 **110 个已被板载外设占用**的引脚
③ 开发板原理图	`_refs/vendor/Davinci_Lite_1V0.pdf` sheet 13 符号几何	**真实**的 `J5` / `J6` 排针 针→球号 表（用 `tools/pdf_text.py --geom` 量取）
④ 模块说明书	`_refs/vendor/HSTAD9238_module_manual.pdf` p9 §6	载板 40 针的**网络名**表（与载板 netlist 互证）
⑤ 待校核约束	`constr/enc_top.xdc`	本项目实际写下的 `PACKAGE_PIN`

★★ 关键发现：厂商 .xls 里的 "J7" 不是本板的 J7

厂商表头的原文是 `双AD9226/AD9238对应管脚 **XC7A35V2.0T**扩展口J6对应管脚 / …J7… / …J11…` —— **这三个口是"XC7A35V2.0T"那块开发板上的扩展口，不是达芬奇-lite 的排针。**

在本板上：`J7` 是 `RGB_TFTLCD` 接口，而且**根本没有 `J11` 这个连接器**（FGG484 上的 `J11` 是 `VCCBRAM` 球号）。

把三套映射按 **Bank** 归类，对应关系立刻清楚：

厂商列	28 个信号落在	本板上是什么	可用?
vendor `J6` 列	**38/38 全在 Bank 14**	以太网 / HDMI / IIC / TF 卡 / LED，**未引出到任何连接器**	✗
vendor `J7` 列	**39/39 全在 Bank 16**	← **只有这一列落在真实排针上**，即本板的 `J6`	✓
vendor `J11` 列	**39/39 全在 Bank 34**	RGB LCD + `V4` (sys_clk)	✗

> ⚠️⚠️ **但是：厂商 `J7` 列的「针号」与本板 `J6` 排针的「针号」一个都对不上（0/28）。** 例如厂商把 `AD2CLK` 放在 `B13`，而本板 `J6` 的第 9 针是 `F16`。 **所以绝不能把厂商表直接抄进 XDC** —— 必须用从本板原理图几何量出来的针→球号。 这正是本工程最终采用 `tools/pdf_text.py --geom` 提取那两张表的唯一原因。

另一处独立矛盾：所谓"直插直用"只对本板成立

模块说明书 p9 §6 自己写明载板 **针 6 / 7 / 8 = 不用**； 而厂商 `J7` 列在这三个位置放的是 `V22`(sd_mosi) / `W22`(iic_scl) / `W21`(uart_txd)。 ⇒ 商家说的"完美适配、直插直用"是针对**厂商自己那块 XC7A35V2.0T 板**的； 插到任何第三方板上都不可能是 1:1 对位，必须按本表接线。

校核脚本输出（`tools/pinout_verify.py`）

```
[A] Ball 存在性              ✓ 50/50 个球都存在于封装内
[B] Bank 归属                ✓ AD9238 组 28 个全在 Bank 16（J6）
                             ✓ 输出组   16 个全在 Bank 15（J5）
                             ✓ 板载 uart_rx(T20)/uart_tx(W21) 均为 Bank 14（2026-10-10 新增 uart_rx）
[C] 重复分配                 ✓ 无重复（50 个端口用了 50 个不同球）
[D] 与板载占用表冲突         ✓ 扩展口引脚 0 个命中（全部空闲；uart_rx/uart_tx 命中板载占用表属预期）
[E] 端口完整性               ✓ 50/50 已约束，IOSTANDARD 全为 LVCMOS33
[F] 排针合法性               ✓ 全部球号都能在 J5/J6 针表里定位
[G] 载板 28 信号全覆盖       ✓ 28/28 逐个一致
RESULT: PASS
```

---

1. 表 A：AD9238 模块 40 针定义

来源：`_refs/vendor/HSTAD9238_module_manual.pdf` + `_refs/vendor/AD9238_carrier_schematic.pdf`（整理于 `docs/AD9238_module_digest.md`）。

模块是 **2×20 = 40 针 2.54 mm 排母**。

模块针号	功能	方向	说明
1	+5V	电源	**不要接反！**
2, 4, 5	GND	电源	
3, 6, 7, 8	未使用	—	悬空
9	CH2_CLK	**FPGA → 模块**	AD9238 的 **CLK_B 输入**
10–18	CH2_D0 … CH2_D8	模块 → FPGA	通道 2 数据低 9 位
19, 20	未使用	—	
21–23	CH2_D9 … CH2_D11	模块 → FPGA	通道 2 数据高 3 位
24	CH2_OTR	模块 → FPGA	通道 2 超量程
25	未使用	—	
26	CH1_CLK	**FPGA → 模块**	AD9238 的 **CLK_A 输入**

模块针号	功能	方向	说明
27–35	CH1_D0 … CH1_D8	模块 → FPGA	通道 1 数据低 9 位
36–38	CH1_D9 … CH1_D11	模块 → FPGA	通道 1 数据高 3 位
39	CH1_OTR	模块 → FPGA	通道 1 超量程
40	未使用	—	

> **模块针号与开发板 `J6` 针号一一对齐**：`模块针 k ⇔ J6 针 k`（排母直插）。 但 `J6` **排针本身是 `Header_2*26` = 52 针**（见 §1.2），模块只占满前 40 针， 41…52 针在本工程中不使用。

1.1 关键事实（已核对 ADI 原厂手册）

事实	说明
AD9238 没有 DCO（数据输出时钟）	只有 `CLK_A` / `CLK_B` 两个**时钟输入**。数据在时钟上升沿后 `tPD = 2…6 ns` 有效，流水延迟 7 拍。
CH1_CLK / CH2_CLK 是 ADC 的采样时钟输入	由 FPGA 驱动，不是模块输出的数据时钟。
`AD1CLK` 与 `AD2CLK` 在载板上是两根独立网络	`AD1CLK` = `J6-26` + `U1-63 CLK_A`； `AD2CLK` = `J6-9` + `U1-18 CLK_B`。**无共用网络、无时钟缓冲器、无板上晶振。** ⇒ **必须用两个 FPGA 引脚分别驱动**，本工程对应 `ad_clk`(B18) / `ad_clk2`(F16) 两个 ODDR 输出。
输出编码	**偏移二进制**：−5 V→`000h`，0 V→`800h`，+5 V→`FFFh`； **+3.3 V CMOS**。
供电	由开发板通过 `J6-1` 供给，说明书 p5 写 "4~9 V，默认 5 V"、p4 写 "4~5.5 V"（**说明书自相矛盾，以 5 V 为准**），约 0.4 A。

所需 IO = 24 根数据 + 2 根独立时钟 + 2 根 OTR = 28 根。

> ★ **载板上只有一颗 AD9238（`U1`）**，A/B 两通道共用同一片硅片， 但 **`CLK_A` 与 `CLK_B` 在板上是两根互不相连的网络**，所以 FPGA 必须出**两个**引脚： 本工程用 `ad_clk`（`B18` → `J6-26`）与 `ad_clk2`（`F16` → `J6-9`）两个 ODDR， 由**同一个 `clk_i`** 驱动 ⇒ 两路相位严格一致，两通道同步采样。 （两个独立输出各自只驱动一根短线，边沿质量也比"一个脚带两根线"更好。）
>
> ★ 载板**无 DCO**： `DCO` 这个词在载板原理图（1606 个文本 span）与说明书（28 页）里**都不存在**。

---

2. 最终接线表（已写入 `constr/enc_top.xdc`）

2.0 两根排针的分工

排针	类型	Bank	电源轨	用途
`J6`	`Header_2*26`（52 针）	**16**	`VCCIO_B16`，跳线 `J10` 选压	AD9238 载板（40 针直插）
`J5`	`Header_2*26`（52 针）	**15**	`VCCIO_B15`，跳线 `J9` 选压	本项目其余 IO（ABZ/UVW/SPI/中断）

`J6` 被载板占满 1…40 针后，只剩 6/7/8、19/20、25、40 这 7 个"不用"位， 放不下 ABZ(6)+SPI(4)+状态灯(2)+中断(1) = 13 根， 而且这几个位置的球号无法从原理图几何中量出（只量到 pins 2–5 与 9–40）， **因此输出侧整体改走 `J5`** —— `J5` 的 48 个信号球经校验全部是 Bank 15 且未被任何板载外设占用。

2.0b 排针针→球号对照（★ 从原理图几何量取，不是从厂商 .xls 抄的）

`J6` 的 1 脚 = `+5V`（**对载板的电源输入**）、3 脚 = `VCCIO_B16`； `J5` 的 1 脚 = `+5V`、3 脚 = `VCCIO_B15`。

```
J6:  9=F16  10=E17  11=C13  12=B13  13=E16  14=D16  15=A13  16=A14
     17=D14  18=D15  21=C14  22=C15  23=B15  24=B16  26=B18  27=F18
     28=E18  29=C17  30=D17  31=A18  32=A19  33=C18  34=C19  35=F20
     36=F19  37=B20  38=A20  39=E19
J5:  5=H20   6=G20   7=M21   8=L21   9=N22  10=M22  11=N20  12=M20
     13=L19  14=L20  15=J22  16=H22  17=J20  18=J21  19=K21  20=K22
     21=M15  22=M16  23=L13  24=M13  25=K13  26=K14  27=J19  28=H19
     29=H14  30=J14  31=H17  32=H18  33=G13  34=H13  35=G16  36=G15
     37=K18  38=K19  39=N18  40=N19  41=L18  42=M18  43=J17  44=K17
     45=K16  46=L16  47=L14  48=L15  49=H15  50=J15  51=G17  52=G18
```

2.1 AD9238 信号（全部在 `J6`，全部 Bank 16，全部 `LVCMOS33`）

载板丝印	模块针 = `J6` 针	FPGA 引脚	`enc_top` 端口
AD1CLK	26	**B18**	`ad_clk`
AD1D0	27	**F18**	`ad_ch1_d[0]`
AD1D1	28	**E18**	`ad_ch1_d[1]`
AD1D2	29	**C17**	`ad_ch1_d[2]`
AD1D3	30	**D17**	`ad_ch1_d[3]`
AD1D4	31	**A18**	`ad_ch1_d[4]`
AD1D5	32	**A19**	`ad_ch1_d[5]`
AD1D6	33	**C18**	`ad_ch1_d[6]`
AD1D7	34	**C19**	`ad_ch1_d[7]`
AD1D8	35	**F20**	`ad_ch1_d[8]`
AD1D9	36	**F19**	`ad_ch1_d[9]`
AD1D10	37	**B20**	`ad_ch1_d[10]`
AD1D11	38	**A20**	`ad_ch1_d[11]`
AD1_OTR	39	**E19**	`ad_ch1_otr`
AD2CLK	9	**F16**	`ad_clk2`（★ 独立第二路时钟）
AD2D0	10	**E17**	`ad_ch2_d[0]`
AD2D1	11	**C13**	`ad_ch2_d[1]`
AD2D2	12	**B13**	`ad_ch2_d[2]`

载板丝印	模块针 = J6 针	FPGA 引脚	enc_top 端口
AD2D3	13	**E16**	`ad_ch2_d[3]`
AD2D4	14	**D16**	`ad_ch2_d[4]`
AD2D5	15	**A13**	`ad_ch2_d[5]`
AD2D6	16	**A14**	`ad_ch2_d[6]`
AD2D7	17	**D14**	`ad_ch2_d[7]`
AD2D8	18	**D15**	`ad_ch2_d[8]`
AD2D9	21	**C14**	`ad_ch2_d[9]`
AD2D10	22	**C15**	`ad_ch2_d[10]`
AD2D11	23	**B15**	`ad_ch2_d[11]`
AD2_OTR	24	**B16**	`ad_ch2_otr`

> ★ **`AD1CLK` 与 `AD2CLK` 是两根独立网络**（见 §1.1），所以本工程用 **两个 ODDR** 分别驱动 `ad_clk`(B18) 与 `ad_clk2`(F16)，各自建生成时钟、各自约束 `set_output_delay`。 **不要再把这两针短接** —— 载板本来就是分开的，短接会让两个输出对打。

2.2 输出侧 IO（全部在 `J5`，全部 Bank 15，全部 `LVCMOS33`）

J5 针号	FPGA 引脚	本项目用途	enc_top 端口
5	**H20**	ABZ 增量输出 A	`a_out`
6	**G20**	ABZ 增量输出 B	`b_out`
7	**M21**	ABZ 增量输出 Z	`z_out`
8	**L21**	UVW 换相 U	`u_out`
9	**N22**	UVW 换相 V	`v_out`
10	**M22**	UVW 换相 W	`w_out`
14	**L20**	SPI 时钟	`spi_sck`
15	**J22**	SPI 片选（低有效）	`spi_ss_n`
16	**H22**	SPI 主机输出/从机输入	`spi_mosi`
17	**J20**	SPI 主机输入/从机输出	`spi_miso`
18	**J21**	状态指示灯 2（本板无 LED，探针用）	`led[2]`
19	**K21**	状态指示灯 3（本板无 LED，探针用）	`led[3]`
20	**K22**	中断输出（低有效，给外部 MCU）	`irq_n`

> 全部 13 根都在 **Bank 15**。量产/加分项：外接 `AM26LV31` / `DS26LV31`（RS-422）差分驱动。

2.3 板载专用引脚（不走扩展口）

信号	FPGA 引脚	Bank	说明
`sys_clk`	**V4**	34	板载 50 MHz 晶振（`V4` **没有**引到任何扩展口）
`key_n`	**W9**	34	用户按键 KEY0（低有效）→ 一键标定
`led[0]`	**AB21**	14	指示 ADC 超量程 `otr_lat`
`led[1]`	**AB22**	14	指示幅值告警 `amp_low|amp_high`
`uart_tx`	**W21**	14	板载 USB 串口（FT2232 的 FPGA 接收侧），2 Mbaud 遥测

> Bank 划分由 Vivado 查询器件封装得到（`_synth_work/pkgpins.csv`）。 本工程用到四个 bank：**Bank 16**（`J6`，需把 `J10` 拨到 3.3 V）、 **Bank 15**（`J5`，需把 `J9` 拨到 3.3 V）、**Bank 34**、**Bank 14**（后两者是板上固定 3.3 V 轨）。

2.4 引脚完整性（决定能否生成 bitstream）

```
python tools/pinout_verify.py
  [E] 端口完整性（enc_top 的端口是否都有 PACKAGE_PIN + IOSTANDARD）
      RTL 端口数 50 ；已约束 50 ；未约束 0
      ✓ 全部端口都有引脚约束 —— 不会触发 UCIO-1
      ✓ 全部端口 IOSTANDARD = LVCMOS33（J6/J5 跳线到 3.3V 后成立）
  RESULT: PASS
```

`enc_top` 的 **50 个端口全部有** **`PACKAGE_PIN` + `IOSTANDARD`**（2026-10-10 加 UART→SPI 桥后新增 `uart_rx` = `T20`、端口数由 49 变 50），因此实现后的 DRC 不会出现 `UCIO-1`（未约束端口），**可以正常生成 bitstream。** `led[2]` / `led[3]` 本板无对应 LED，为消除 `UCIO-1` 已引到 `J5-18(J21)` / `J5-19(K21)`， 可用示波器或外接 LED 观察状态。

3. 上电前检查清单（逐项打勾，不要跳）

3.1 断电状态

- [ ] 载板插在达芬奇-lite 的 **`J6`**（`Header_2*26`，52 针，bank 16）。 ⚠️ **不是 `J6` / `J7` / `J11` 这三个厂商 .xls 列名**——那是另一块 "XC7A35V2.0T" 板的扩展口； 本板的 `J7` 是 `RGB_TFTLCD`，且本板**根本没有 `J11` 连接器**。详见 §0.1。
- [ ] 载板**没有插反**（模块 40 针排母的 pin 1 与 `J6` 排针的 pin 1 对齐； `J6-1` 是 `+5V`， `J6-3` 是 `VCCIO_B16`）
- [ ] 万用表确认：模块针 1（+5V）与针 2/4/5（GND）**不短路**
- [ ] 万用表确认： **`J6-1` (+5V) 与 `J6-2` / `J6-4` (GND) 的极性正确**
- [ ] ★ 万用表确认： **`AD1CLK` 与 `AD2CLK` 在载板上是两根独立网络（不相通）**。 本工程用 `ad_clk`（`B18`）与 `ad_clk2`（`F16`）两个 FPGA 引脚**分别**驱动它们； 若量到相通，说明模块型号与说明书 §6 不符，须停下来核对，**不要短接。**
- [ ] ★ **两个 bank 的电压跳线都拨到 3.3 V**： `J10` 选 `VCCIO_B16`（模块所在 bank）、 `J9` 选 `VCCIO_B15`（输出侧 bank）。 两者都由可调 LDO `U22`（`SPX3819M5-L-TR`， `VIN=+5V`）经三位跳线 `MINI_BOOT_3*2` 选

择， **出厂不保证是 3.3 V**；选 3.3 V 时 `R1=79K`（`R139`）。若实际不是 3.3 V， `LVCMOS33` 约束会触发 "Bank VCCO 与 IOSTANDARD 不符" 的 DRC，或直接损坏 IO。

- [ ] 所有 FPGA IO **在 bit 文件里都已约束**（实现后 `report_drc` 应无 `UCIO-1`）
- [ ] 信号源输出幅度设在 **10 Vpp / 高阻**（不是 1 Vpp），DC 偏置 0 V
- [ ] 信号源与 FPGA 板**共地**
- [ ] ★ **模拟输入走 SMA，不走杜邦线**：**两根 50 Ω BNC→SMA 同轴**，CH1 芯线→载板 `J1`/`CH1`、CH2 芯线→载板 `J2`/`CH2`，两条屏蔽随同轴线落在载板地
- [ ] ★ **两路各设 10 Vpp、0 V 偏置**（不是"一端 10 Vpp + 一端反相"——那等效 20 Vpp 会切顶）；**两路相差 90°**
- [ ] 万用表量 `J1`/`J2` SMA 芯线对地 ≈ **4 kΩ**（`R13`/`R5` = 4.02 K 分压电阻），既不是 0 Ω 也不是开路

> ★ 你手上的信号源是**双端（两路独立单端、共地）输出**，载板每个 SMA 通道内部是 **单端输入 → AD8065(×1/5.02) → AD8138(转差分) → AD9238**，所以**不需要**从外部送差分进 SMA。 完整接线图、信号源逐项参数、相位方向说明与两种确认方法见 `docs/06_board_test_guide.md` §1.3。

3.2 分级上电（这一步必须分三级，不要一次全插）

第 1 级：只给载板供电 1. 只接 +5V 和 GND（不接任何信号线），给开发板上电。 2. 观察电流（应约 0.4 A），摸载板温度（应微温，不烫）。 3. 若发烫或电流异常 → 立刻断电，检查是否插反。 4. 用示波器看载板的 CH1_CLK/CH2_CLK 引脚（此时应无信号）确认没有意外驱动。

第 2 级：接数据线与时钟 5. 断电，接上 24 根数据线（A 组 12 + B 组 12）+ **2 根独立时钟**（`AD1CLK` ← `ad_clk`/`B18`， `AD2CLK` ← `ad_clk2`/`F16`）+ 2 根 OTR。 6. 下载一个**只输出时钟、不读数据**的 bit 文件（或直接下 `_synth_work/enc_top.bit`，只是先不接信号源）。 7. 示波器看载板的两个输入点： `AD1CLK` 与 `AD2CLK` 上应有**同相**的干净 50 MHz、50% 占空、3.3 V 摆幅方波。

> 两路是同一 ODDR 结构推出来的， `-invert` 后相位一致；若看到两路反相，检查是否只约束了其中一路。 8. 若波形变形/振铃严重 → 缩短线长（**建议 ≤5 cm**），或在时钟线上串 22~33 Ω 端接电阻。

第 3 级：读数据 9. 保持 `enc_top.bit` 在板上（**本机没有 ILA 可用**：Vivado BASIC 许可证不支持 `create_debug_core`，见 `docs/06` §2.2）。 内部观测走 **UART 遥测**： `python tools\uart_scope.py --port COM7 --baud 2000000 --seconds 5 --png adc.png`。 10. 信号源给 1 kHz / 10 Vpp 正弦（CH1）、同频 90° 相差（CH2）。 11. UART 的 `s_n`/`c_n` 两列应看到两路**相位差 90° 的 Q1.15 正弦**（原始码是偏移二进制、中心在 `0x800`， 可在载板数据脚上用示波器直接量）。 12. **此时是最关键的一步**：扫捕获时钟相位，以及采样率 25 / 40 / 50 MHz，记录哪一组最稳定（连续观察 10 分钟无跳变）。 13. 若所有相位都不对：检查 `ad_clk`/`ad_clk2` 是否真的接到了载板的 CLK 脚；检查数据线是否错位（用 §3.3 逐位验证法）。

3.3 数据线逐位验证技巧（强烈推荐）

> 不要一次接 24 根线然后祈祷。**先只接 `CH1_D0` 一根，验证它对应 `ad_ch1_d[0]`，再一根一根加。** 方法（两条路一起用）： ① **示波器直接量载板的数据脚** —— 信号源给缓慢三角波（如 1 Hz）时， `CH1_D0` 应是 50 % 占空方波、 `CH1_D1` 频率减半……第 k 位的翻转频率是第 0 位的 1/2^k； ② **UART 快查** —— `uart_scope.py --csv adc.csv` 导出后看 `s_n` 是否单调爬升再回落；某一位恒定或反相就说明那条线接错了。 **这个笨办法能在 30 分钟内替代 3 天的"数据不对"排查。**

---

4. 上电后的功能验证（李萨如图法）

操作	李萨如图（`s_n − c_n`，用 `uart_scope.py --png lis.png` 或示波器 XY 模式画 CH1-CH2）	反映的误差
理想情况	正圆、圆心在原点	无误差
加 0.2 V 直流偏置	圆心偏离原点	直流偏置误差
把 CH2 幅度从 10 Vpp 降到 8 Vpp	变成斜椭圆	幅值失衡
把相位差从 90° 改成 85°	变成斜椭圆并旋转	正交相位误差
按"一键标定"	圆变正圆，误差掉到 0.02° 量级	**六参数闭式标定的效果**

5. 常见故障速查表

现象	最可能的原因	排查动作
UART 的 `s_n` / `c_n` 恒为 0 或恒为满量程	时钟没到载板 / 载板没供电	示波器量载板 CLK 引脚；量 +5V
数据随机跳变	捕获相位不对 / 线太长 / 没共地	扫相位 0/90/180/270；缩短线；共地
数据看起来像正弦但幅度只有几十个 LSB	信号源幅度太小（1 Vpp）	改成 10 Vpp

现象	最可能的原因	排查动作
两通道波形相位差不是 90°	载板不是"共用一根时钟"型：`AD1CLK` / `AD2CLK` 是两根独立网络，必须两根都接上（`B18` → `AD1CLK`、`F16` → `AD2CLK`）	示波器分别量 `AD1CLK` 与 `AD2CLK` 两个输入点，两处都要有 25/50 MHz 方波；只接一根会导致其中一路无输出
某一位恒定不变	该数据线没接或接错	用 §3.3 的逐位验证法
`OTR` 一直为 1	输入超过 ±5 V	减小信号源幅度
角度输出在 ±180° 附近跳变	象限折叠逻辑写错	用黄金模型比对；检查第二象限是否写成 `x'=+y`
角度误差周期性波动 ~20°	没做标定 / 标定系数没写入	触发一键标定（按 `KEY0`，或 SPI 写 `0x0502=0x0001`）；用 SPI 读回 `SC_OFS_S` 等寄存器

现象	最可能的原因	排查动作
AB 输出抖动	迟滞太小 / PI 滤波 kp 太小	**★ 2026-10-11 上板定案（见 `DEV_STATUS.md` §1.40，本条已按实测修正）：真实旋钮是环路 I 项增益 `0x0302`** —— 板上默认 `0x0302 = 0x1C`（kp4/ki3）时 A/B 跳变是理想值的 **2.7×**（57 vs 20.8，自由运行 stage1 窗口）；**写 `0x0302 = 0x0004`（kp4/ki0，纯 P）⇒ 1.0×（20 = 理想）、零反向点火、间隔 44/48/52 clk 近均匀，跟踪残差也最小**；`0x0302 = 0x000C`（ki=1）⇒ 1.1×。⚠️ **"加大 AB 迟滞"已实测无效**（统一窗口复测 hyst 0/2/4/8 ⇒ 2.7×/3.1×/3.9×/3.9×，单调变差）⇒ §1.38 曾推荐的 `0x0103 = 0x0002` **作废。**★ 纯 P 是 I 型环路，高速档要量跟踪误差（滞后 ≈ 速度÷(1/64)：1 kHz≈0.115°、10 kHz≈1.15°）。要"开机即纯 P"须改 `rtl/reg_file.v:280` 复位值（v7 重综合）。**★★ 2026-10-11 晚追加（v8，见 `DEV_STATUS.md` §1.43）：v7 的"纯 P 默认"已被取代** —— v8 打开 `GATE_STATE`（环路状态回到 per-sample 语义）并把默认改为 **kp=2 / ki=3（`0x0302 = 0x1A`）**，全链路 A/B 回放同时满足"跳变 = 理想、|d| ≤ 11 LSB、0 次方向反转"（对比只开门控仍 kp4/ki3：跳变 126 但 |d| 冲到 3331 LSB ≈ 1.8°、72 次反转 ⇒ 门控必须与 kp/ki 成套重调）。⇒ 上板后应读回 `0x0302 = 0x1A`；`0x0004` 纯 P 只作对照档。~~v8 位流尚未重综合~~ → **v8 早已重综合并上板（§1.45，读回 `0x05F2` = 8）；现行位流是 v12（2026-10-13，`0x05F2` = `0x000C`、`0x0302` 仍为 `0x1A`）**
下载 bit 后 FPGA 就"死"	JTAG 时钟太快	把 JTAG/下载器频率降到 ≤10 MHz
综合/实现报时序不收敛	长组合路径	检查是否每级都打拍
实现报 `UCIO-1` / 无法生成 bitstream	有端口没有 `PACKAGE_PIN`	跑 `python tools/pinout_verify.py` 看 §E。**本版已 50/50 全部分配并成功生成 bitstream**，所以正常情况下不会再出现；若你改过 XDC 后复现，说明改动引入了新的未约束端口
报 "Bank 16 / Bank 15 VCCO 与 IOSTANDARD 不符"	`J10`(VCCIO_B16) 或 `J9`(VCCIO_B15) 跳线不在 3.3 V	按 §3.1 把两个跳线都拨到 3.3 V，或把该 bank 的 IOSTANDARD 改成实际 VCCO

现象	最可能的原因	排查动作
输出侧（ABZ 六路 / SPI / `led[2:3]` / `irq_n`）无波形	这些脚在 **`J5`**（bank 15），不是 `J6`	按 §2.2 的 `J5-5…J5-20` 表接线